

# Job-Hunting Assistant

## MVP and v1 Technical Planning Document

*Founder-facing product, architecture, and delivery blueprint*

|                        |                                                                                                                             |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Document purpose       | Translate the product brief into a practical technical plan for an MVP and a realistic v1.                                  |
| Primary recommendation | Build an individual-focused job-search operating system around job aggregation, explainable matching, and workflow support. |
| Build posture          | Use a modular monolith with background workers, official/public data integrations, and selective AI assistance.             |

Prepared from the supplied product brief.

## 1. Executive Summary

The product should be built as an individual-focused job-search operating system rather than an auto-apply bot. The MVP should help a serious job seeker discover higher-fit roles faster, ignore noise, stay organized, and make better decisions with less manual overhead.

The recommended technical posture is a modular monolith with strong internal boundaries, background workers, a connector framework, and a canonical job catalog. This provides speed to first value, simpler operations, and a clean path to future expansion without premature microservices.

The core differentiators should be multi-source aggregation from official or public sources, deduplication and merged job views, explainable per-user fit scoring, unified search plus tracking, and application support that assists the user without taking control away from them.

## 2. Product Scope

### In scope for MVP

- User account, profile, preferences, and resume upload.
- Structured profile extraction from the user's resume.
- Job ingestion from a limited initial set of official or public sources.
- Canonical job model, normalization pipeline, and duplicate handling.
- Search, filtering, saving, hiding, bookmarking, and saved searches.
- Explainable user-specific fit scoring for each canonical job.
- A lightweight application tracker with statuses, notes, tasks, and reminders.
- Email and in-app notifications for high-fit jobs and follow-through.
- Resume and application preparation support such as keyword gaps and talking points.

### In scope for v1

- More connectors and better source health automation.
- Improved ranking using behavioral feedback and semantic similarity.
- Better company enrichment, role closure detection, and smarter alerts.
- Richer resume drafting and export support.
- Improved duplicate clustering and personalization.

### Explicit non-goals for MVP

- Autonomous mass auto-apply behavior.
- Recruiter CRM or employer-side tooling.
- Social networking or community features.
- Unofficial scraping-heavy foundation or arbitrary browser automation.

## 3. Primary User Personas

### Focused professional job seeker

A user with a clear target role who wants higher quality filtering, faster triage, and a way to stay organized through the process.

### **Career-transition job seeker**

A user shifting into a new domain, title, or seniority band who needs transferable-skill interpretation, stretch-role handling, and concrete explanation of gaps.

### **High-volume but selective applicant**

A user reviewing many postings who still wants decision quality, duplicate suppression, and a lightweight process tracker.

## **4. User Journeys and Key Workflows**

### **Initial setup**

- User signs up and uploads a resume.
- System extracts structured profile data and inferred skills.
- User edits target roles, salary floor, locations, and deal breakers.
- System starts ingesting and scoring jobs for that profile.

### **Discover and triage jobs**

- User opens the Top Matches view.
- Feed shows fit score, strengths, gaps, and basic explanation.
- User filters, saves, hides, or shortlists jobs.
- User feedback tunes future ranking.

### **Evaluate a job deeply**

- User opens a normalized job detail page.
- System surfaces required versus preferred qualifications, urgency, salary, duplicates, and source details.
- User decides whether to archive, save, or prepare an application.

### **Prepare to apply**

- System suggests relevant resume bullets, missing keywords, and cover letter talking points.
- User selects or uploads the resume version to use.
- Tracker status moves to ready to apply or applied.

### **Stay on top of progress**

- System sends digests, stale reminders, and follow-up prompts.
- User updates status, tasks, and notes from a single workflow surface.

## **5. Functional Requirements**

### **User profile and preferences**

- Account creation and authentication.
- Resume upload and storage.
- Structured editing of titles, industries, skills, salary minimum, locations, work preferences, visa constraints, deal breakers, and stretch-role intent.

### **Job ingestion**

- Scheduled sync for each source connector.
- Persistence of raw source payloads.
- Normalization into a canonical internal schema.
- Handling of partial or inconsistent fields and source-specific metadata.

### **Matching and ranking**

- Per-user fit score, strength summary, gap summary, deal breakers, and confidence band.
- Re-scoring when user preferences change or job records are updated.
- Feedback-aware signals for future ranking improvements.

### **Search and discovery**

- Browse, search, filter, sort, save searches, bookmark, hide, and mark not relevant.

### **Application workflow**

- Pipeline statuses from discovered to archived.
- Notes, tasks, reminders, timestamps, and associated resume versions.

### **Resume and application support**

- Resume tailoring suggestions, keyword gap analysis, relevant bullet suggestions, and cover letter talking points.

### **Notifications**

- Daily or weekly digests, high-fit alerts, follow-up reminders, and closure or disappearance alerts when detectable.

## **6. Non-Functional Requirements**

- **Performance:** most feed and detail views should feel fast under normal load, with target response times under two seconds for common user actions.
- **Reliability:** connector failures must be isolated so one failing source does not break the rest of the system.
- **Scalability:** the architecture should comfortably support early growth from thousands of users and hundreds of thousands of jobs.
- **Security and privacy:** sensitive career data must be encrypted and handled with least-privilege access.
- **Maintainability:** connector contracts, normalization rules, and scoring versions should be explicit and replayable.

- Explainability: score outputs must be reconstructable and auditable rather than opaque.

## 7. MVP Scope vs Post-MVP Scope

| Area           | MVP                                    | Post-MVP / v1                                        |
|----------------|----------------------------------------|------------------------------------------------------|
| Connectors     | 5 to 7 high-value sources.             | 10 to 20 plus sources over time.                     |
| Search         | Structured keyword search and filters. | Semantic and hybrid search.                          |
| Ranking        | Rules and heuristics.                  | Behavior-aware and embedding-assisted ranking.       |
| Deduplication  | Deterministic plus fuzzy rules.        | Graph or clustering based duplicate resolution.      |
| Resume support | Suggestions and talking points.        | Tailored draft generation and richer export support. |
| Notifications  | Digests and reminders.                 | Adaptive tuning and smarter prioritization.          |

## 8. System Architecture Proposal

The recommended MVP architecture is a modular monolith backend paired with background workers. The API handles user-facing workflows while asynchronous jobs handle ingestion, parsing, deduplication, scoring, and notifications.

- Frontend: a web application for profile setup, discovery, job detail, tracking, and notification settings.
- Backend API: modules for auth, user profile, jobs, search, ranking explanations, tracker, and resume support.
- Workers: background processing for connector sync, normalization, scoring, reminder generation, and source health checks.
- Storage: PostgreSQL as system of record, Redis for queues and caching, and object storage for resumes and generated files.

## 9. Recommended Services and Modules

- Identity and account management
- User profile and preferences
- Resume and document management
- Resume extraction and enrichment
- Connector framework
- Ingestion orchestration
- Canonical job catalog
- Deduplication service
- Matching and ranking engine
- Search and query service

- Application tracker
- Notification service
- Feedback and personalization
- Admin and operational console

## 10. Data Model and Core Entities

The following entities should form the core data model. Source-specific data should be preserved without polluting the canonical model.

### **User**

Account identity, email, timezone, and auth provider.

### **UserProfile**

Structured profile including current title, years of experience, work authorization, and transition notes.

### **UserPreference**

Preferred titles, industries, skills, locations, salary thresholds, remote preferences, and deal breakers.

### **Resume**

Stored file reference, parsed text, parser version, and metadata.

### **ResumeStructuredProfile**

Normalized skills, roles, companies, industries, certifications, and extraction confidence.

### **JobSource**

Source name, source job id, raw payload, fetch metadata, and source status.

### **CanonicalJob**

Normalized title, company, description, location, remote type, compensation, seniority, and job status.

### **JobRequirement**

Required or preferred qualifications broken into typed records.

### **JobSourceMapping**

Mapping between source jobs and a canonical job record.

### **JobDuplicateGroup**

Duplicate cluster metadata and merge confidence.

### **UserJobScore**

Per-user score, confidence, strengths, gaps, and scoring version.

**UserJobState**

User actions such as saved, hidden, shortlisted, or archived.

**Application**

Tracked application status, timestamps, notes, and documents used.

**TaskReminder**

Reminders and follow-up tasks associated with a job or application.

**FeedbackEvent**

Thumbs up, thumbs down, less-like-this, or other preference feedback.

## 11. Connector and Integration Architecture

Each connector should implement a common contract and isolate source-specific behavior behind adapters. The system should prefer official or public APIs, ATS job feeds, and legitimate access paths instead of scraping-first methods.

- Connector contract should include fetch, normalize, change detection, rate-limit policy, and health check behaviors.
- The ingestion pipeline should persist the raw payload first, then normalize into a source-normalized shape, then map into canonical jobs.
- Connectors should be independently schedulable and individually observable.
- Recommended initial MVP connectors are Greenhouse, Lever, Ashby, USAJOBS, SmartRecruiters, Workable, and one broader aggregator such as Adzuna or Jooble.

## 12. Matching and Ranking Engine Design

The first scoring system should be explainable, weighted, and deterministic enough to support user trust. It should combine structured user preferences with extracted job attributes rather than relying on opaque freeform model output.

| Score component                       | Indicative weight |
|---------------------------------------|-------------------|
| Title similarity                      | 20%               |
| Skill overlap                         | 20%               |
| Seniority and experience fit          | 15%               |
| Location and remote fit               | 10%               |
| Compensation fit                      | 10%               |
| Domain or industry relevance          | 10%               |
| Required qualifications satisfaction  | 10%               |
| Stretch-role or trajectory adjustment | 5%                |

- Hard penalties or deal breakers should capture salary floor mismatches, sponsorship conflicts, impossible location constraints, and missing must-have qualifications when the user marks them as non-negotiable.

- Every score should persist component-level reasoning, strengths, gaps, penalties, confidence, and scoring version.
- Later versions can add behavior-aware weighting, embeddings for semantic similarity, and better confidence calibration.

### 13. Deduplication and Normalization Strategy

Duplicate management is critical because repeated postings waste time and damage user trust. The system should aggressively normalize titles, company names, locations, remote types, salary units, seniority labels, and description text before duplicate decisions are made.

- Use deterministic matches first: same source job id, same application URL, same ATS req identifier, or near-identical normalized records.
- Use fuzzy candidate matching second: normalized company similarity, title similarity, location similarity, description similarity, and posted-date proximity.
- Use confidence thresholds: high-confidence candidates auto-merge, medium-confidence candidates soft-cluster, and low-confidence candidates remain separate.
- Expose a merged job view that shows alternate sources while keeping traceability to each source record.

### 14. Search and Filtering Architecture

For MVP, PostgreSQL should handle structured search, filtering, and sorting. This keeps the system operationally simple while still supporting full-text search, trigram matching, and indexed filters.

- Create a query-friendly projection such as a `job_search_document` view or table.
- Support keyword search across title, company, and description text.
- Support filters for source, location, remote type, salary, date posted, seniority, sponsorship, and company.
- Support sorting by fit, recency, salary, and match strength.
- Defer OpenSearch or Elasticsearch until semantic or high-scale search becomes a real need.

### 15. Notification Architecture

Notifications should emphasize quality over volume. The product problem includes noisy alerts, so the notification system must be selective and preference-aware.

- Support daily digest, weekly digest, high-fit alert, saved-job reminder, follow-up reminder, and tracked-role closure alert.
- Deliver through email and an in-app notification center in MVP.
- Run candidate generation through eligibility checks, suppression rules, and quiet-hours preferences before delivery.
- Track sends, opens, clicks, and dismissals for future tuning.



## 16. Security and Privacy Considerations

- Encrypt resumes and sensitive application or preference data at rest.
- Use signed object access and short-lived user sessions.
- Minimize sensitive data sent to external AI providers and redact where possible.
- Provide account deletion and data export paths.
- Use least-privilege access for internal tools and maintain audit logs around sensitive operations.

## 17. Observability and Operational Considerations

The system should be measurable from day one. Operational visibility is important because source reliability, parsing quality, and scoring freshness are central to user trust.

- Track connector success rates, sync duration, parse failures, and source freshness.
- Track queue latency, scoring backlog, notification delivery success, and slow query rates.
- Maintain structured logs with correlation identifiers and PII-aware logging discipline.
- Provide internal admin views for connector health, job replay, dedupe inspection, and scoring explanation inspection.

## 18. Risks, Tradeoffs, and Unknowns

- Source inconsistency and schema drift are inevitable. Mitigation: raw payload retention, test fixtures, and connector-specific error handling.
- Ranking quality may be useful before it is truly great. Mitigation: explanation-first UX, conservative confidence labeling, and structured feedback loops.
- Incorrect duplicate merging can erode trust. Mitigation: high merge thresholds and visible traceability.
- Notification fatigue can reduce engagement. Mitigation: digest-first defaults and suppression rules.
- Overuse of LLMs can create fragile or expensive systems. Mitigation: keep core logic deterministic and use AI only where it clearly adds value.

## 19. Recommended Tech Stack

### Frontend

Next.js with TypeScript, Tailwind CSS, and TanStack Query.

### Backend

NestJS on Node.js for a structured modular monolith.

### Database

PostgreSQL with JSONB, full-text search, and trigram indexes.

### Queue and cache

Redis with BullMQ or a similar job framework.

### Object storage

S3-compatible storage for resumes and generated files.

### Email

Postmark, SES, or SendGrid.

### Observability

OpenTelemetry, Sentry, and either Datadog or a Grafana/Prometheus stack.

### Hosting

Managed frontend hosting plus container-based API and worker deployment.

### AI layer

Single provider abstraction used only for extraction, explanation formatting, and application-assist features.

## 20. Suggested Delivery Phases and Roadmap

| Phase   | Objective                      | Key deliverables                                                                                 |
|---------|--------------------------------|--------------------------------------------------------------------------------------------------|
| Phase 0 | Foundation                     | Auth, profile, resume upload, queue infrastructure, connector skeleton, and admin health basics. |
| Phase 1 | Core ingestion and search MVP  | Initial connectors, canonical job schema, normalization, dedupe v1, and searchable feed.         |
| Phase 2 | Explainable matching MVP       | Scoring engine v1, top matches, score explanations, and feedback capture.                        |
| Phase 3 | Tracker and notification MVP   | Pipeline statuses, notes, reminders, digests, and follow-up support.                             |
| Phase 4 | Resume and application support | Keyword gaps, relevant bullet suggestions, talking points, and resume version association.       |
| Phase 5 | v1 upgrades                    | More connectors, smarter ranking, richer alerts, better dedupe, and semantic search evolution.   |

## 21. What to Build First, Second, and Third

### First: ingestion plus canonical job catalog

Build the connector framework, the first small set of integrations, the raw-payload store, canonical job schema, normalization flow, and dedupe v1. Without this layer, the rest of the product has no durable foundation.

**Second: explainable matching plus discovery UX**

Build user preferences, resume extraction, scoring, explanations, feedback controls, and the Top Matches experience. This is the main product differentiator and the point where the product starts to feel intelligent.

**Third: application tracker plus workflow support**

Build statuses, notes, reminders, follow-up flows, digests, and resume version tracking. This makes the product sticky and translates discovery into real job-search progress.

**22. Open Questions Needing Product Decisions**

- Should MVP optimize for narrower high-quality matches or broader coverage?
- Should salary floors and sponsorship constraints act as hard filters or configurable soft penalties?
- Should notifications default to digest-only for new users?
- Should users manage multiple resume variants from day one?
- Should the system support manual job import by URL or clipboard in MVP?
- How should stretch roles influence ranking relative to safe-fit roles?
- What connector mix best fits the initial target market: startup tech, public sector, general knowledge work, or a hybrid strategy?

**Proposed MVP Architecture Summary**

Recommended final summary: a Next.js web app backed by a modular NestJS API, PostgreSQL, Redis-backed workers, S3-compatible document storage, and a connector framework that ingests official or public job data into a canonical job catalog with explainable per-user scoring and lightweight workflow support.